

1 KNN

1.1 The Very Basics

Learning: find a model f that maps from inputs to outputs given a labeled training dataset.

Inference: use a model f to predict an output y given a new example $\vec{x}$

A model is a function f mapping input $\vec{x}$ to output prediction y .

A model family is a set of models that share the same structure but differ in parameters (e.g. layers in a neural network)

Accuracy is calculated as follows:

$$\frac{\sum_{i=1}^N \mathbb{I}[t^{(i)} = y^{(i)}]}{N}$$

A **parameter** is the internal variables of a model learned from data whilst training.

A **hyperparameter** are the settings chosen before training that controls how the learning process works.

Training vs. validation vs. test

Parametric models fix the parameter count. Non-parametric models don't. $\mathcal{O}(1)$ space complexity with respect to the feature count, and never $\Omega(N)$. Hyperparameters are out of this scope.

1.2 KNN

For 1NN: Find $t^{(c)}$ that corresponds to:

$$\vec{x}^{(c)} = \arg \min_{\vec{x}^{(i)} \in \text{training set}} \text{distance}(\vec{x}^{(i)}, \vec{x})$$

For kNN, for the top k other datapoints closest to $\mathbf{x}$, choose the majority class.

Also good to normalize values:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

Advantages:

- Simple
- No training
- Works for reg. and class.
- Adapts easily to new data

Limits:

- Bad w/ high dims
- Sensitive to scaling. Though can be normalized
- Slow for large datasets, time proportional to data
- Must store logs, proportional to data

If k is even, pick a tiebreaker. For this, you should make sure k is odd.

2 Decision Trees

Large trees overfit, small trees underfit. For inference, trace the tree.

Entropy

$$H(X) = - \sum_{x \in X} p(x) \log_2 p(x)$$

Conditional Entropy

$$H(X | Y = y)$$

$$= - \sum_{x \in X} p(x | Y = y) \log_2 p(x | Y = y)$$

Expected conditional entropy

$$H(X | Y)$$

$$= - \sum_{y \in Y} p(y) \sum_{x \in X} p(x | y) \log_2 p(x | y)$$

Information gain

$$IG(X | Y) = H(X) - H(X | Y)$$

Properties

- Entropy nonnegative
- $H(Y | X) \leq H(Y)$
- Y, X are independent $\Rightarrow IG(Y | X) = 0$
- Full determinant rids it

2.1 Decision Tree Algorithm

Keep splitting until:

- All examples have the same label.
- When there are no features left.
- When a depth limit is reached. (hyperparameter)
- When the loss stops decreasing enough. (hyperparameter)

3 Linear / GD

Loss function measures how off once training example is.

Cost functions sum up all.

$$\mathcal{L}(y, t) = \frac{1}{2}(y - t)^2$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y^{(i)}, t^{(i)})$$

For the linear model $\mathbf{y} = W\mathbf{x}$, the **cost** function is:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x} - t^{(i)})^2$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} (X\mathbf{w} - \mathbf{t})^T (X\mathbf{w} - \mathbf{t})$$

3.1 Gradient of the Cost Function

$$\frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

$$\nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w}) = \frac{1}{N} X^T (X\mathbf{w} - \mathbf{t})$$

The regularizer:

$$\frac{\lambda}{2} \sum_{j=0}^D w_j^2$$

The gradient of the regularizer:

$$\frac{\partial}{\partial w_j} \left(\frac{\lambda}{2} \sum_{j=0}^D w_j^2 \right) = \lambda w_j$$

$$\nabla_{\mathbf{w}} \left(\frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \right) = \lambda \mathbf{w}$$

In the event the bias term is left in:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} (X\mathbf{w} + b - \mathbf{t})^T (X\mathbf{w} + b - \mathbf{t})$$

$$\nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w}) = \frac{1}{N} X^T (X\mathbf{w} + b - \mathbf{t})$$

$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)}) x_j^{(i)}$$

$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial b} = \frac{1}{N} (X\mathbf{w} + b - \mathbf{t})$$

3.2 Stochastic GD

Batch GD is what GD normally does. It can be slow.

Stochastic GD just randomly picks a training example. It can be very noisy. Where i at random,

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$$

Mini-batch GD falls in-between.

Batch size can be tuned as a hyperparameter

3.3 Vectorizing Tips

Where X is $N \times D$ and all other variables can have their shape inferred:

Dot product

$$\sum_{i=1}^N u_i v_i = \mathbf{u}^T \mathbf{v}$$

Hadamard product

$$\mathbf{u} \odot \mathbf{v} = \text{diag}(\mathbf{u}) \mathbf{v} = (\mathbf{u}^T \text{diag}(\mathbf{v}))^T$$

Sum a vector

$$\mathbf{u}^T \mathbf{1} = \sum_{i=1}^N u_i$$

Summing a matrix on axis 1 – horizontal squish $> - <$ (shape N)

$$X \mathbf{1}$$

Summing a matrix on axis 0 – vertical squish $\downarrow$ (shape D)

$$\mathbf{1}^T X$$

This is a column vector, and you may need to transpose it.

Row-wise multiplication

$$\text{diag}(\mathbf{u}) X = \begin{bmatrix} - & \mathbf{x}^{(1)} u^{(1)} & - \\ - & \mathbf{x}^{(i)} u^{(i)} & - \\ - & \mathbf{x}^{(N)} u^{(N)} & - \end{bmatrix}$$

Column-wise multiplication

$$X \cdot \text{diag}(\mathbf{v}) = \begin{bmatrix} | & | & | \\ \mathbf{x}_1 v_1 & \mathbf{x}_2 v_2 & \mathbf{x}_3 v_3 \\ | & | & | \end{bmatrix}$$

Tip: in a chain of matrix multiplications, the **height** of the leftmost matrix will be the height of the output. E.g. for $X^T \mathbf{r}$, output is $D \times 1$.

$$\sum_{i=1}^N r^{(i)} \mathbf{x}^{(i)} = X^T \mathbf{r}$$

Shape reasoning: the summed index (N) must become the *inner* dimension and cancel; the dimension that appears first in the result goes on the left.

Stack predictions: $(X\mathbf{w})_i = \mathbf{w}^T \mathbf{x}^{(i)}$, so $X\mathbf{w} \in \mathbb{R}^{N \times 1}$ stacks all predictions.

Scalar with vector ($s \in \mathbb{R}$)

$$\alpha_i = u_i + s \Rightarrow \boldsymbol{\alpha} = \mathbf{u} + s \mathbf{1}$$

$$\beta_i = s u_i \Rightarrow \boldsymbol{\beta} = s \mathbf{u}$$

Outer product

$$M_{ij} = u_i v_j \Rightarrow M = \mathbf{u} \mathbf{v}^T \quad (P \times Q)$$

Inner product $\rightarrow$ scalar; element-wise $\rightarrow$ vector; outer $\rightarrow$ matrix.

Weighted sum of vectors (M rows $\mathbf{m}_i^T$ vs. columns $\mathbf{m}_j$)

$$\sum_i u_i \mathbf{m}_i = M^T \mathbf{u} \text{ (rows)}$$

$$\sum_j u_j \mathbf{m}_j = M \mathbf{u} \text{ (cols)}$$

Stack of dot products (M rows $\mathbf{m}_i^T$)

$$\alpha_i = \mathbf{m}_i^T \mathbf{u} \Rightarrow \boldsymbol{\alpha} = M \mathbf{u}$$

Two weights per term

$$\sum_i u_i v_i \mathbf{m}_i = M^T (\mathbf{u} \odot \mathbf{v}) = M^T \text{diag}(\mathbf{u}) \mathbf{v}$$

Double sums

$$\sum_i \sum_j u_i v_j m_{ij} = \mathbf{u}^T M \mathbf{v}$$

$$\sum_i \sum_j u_i v_j m_{ij}^2 = \mathbf{u}^T (M \odot M) \mathbf{v}$$

$$\sum_i \sum_k u_i m_{ik} u_k = \mathbf{u}^T M \mathbf{u}$$

Pitfall: $\mathbf{a} \odot \mathbf{b} \neq \mathbf{a} \text{diag}(\mathbf{b}) - (P \times 1)(P \times P)$ is undefined; use $\text{diag}(\mathbf{b})\mathbf{a}$.

Pitfall: $\frac{1}{N}(\mathbf{X}\mathbf{w} - \mathbf{t})^T \mathbf{X}$ is a $1 \times (D+1)$ row vector; the gradient must be a column $\Rightarrow \frac{1}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{t})$.

Variants of LR gradient ($\mathbf{r} = \mathbf{X}\mathbf{w} - \mathbf{t}$)

- Per-weight λ_j : $\frac{1}{N} \mathbf{X}^T \mathbf{r} + \boldsymbol{\lambda} \odot \mathbf{w} = \frac{1}{N} \mathbf{X}^T \mathbf{r} + \text{diag}(\boldsymbol{\lambda})\mathbf{w}$
- Per-example weight α_i : $\sum_i \alpha_i r^{(i)} \mathbf{x}^{(i)} = \mathbf{X}^T \text{diag}(\boldsymbol{\alpha})\mathbf{r}$
- Per-feature scale α_j : predictions $\mathbf{X} \text{diag}(\boldsymbol{\alpha})\mathbf{w}$, gradient gains a $\text{diag}(\boldsymbol{\alpha})$ on the left: $\frac{1}{N} \text{diag}(\boldsymbol{\alpha}) \mathbf{X}^T \mathbf{r}'$, $\mathbf{r}' = \mathbf{X} \text{diag}(\boldsymbol{\alpha})\mathbf{w} - \mathbf{t}$

Absorb bias: $\mathbf{w}_1^T \mathbf{x}_1 + b = \mathbf{w}_2^T \mathbf{x}_2$ with $\mathbf{w}_2 = \begin{bmatrix} b \\ \mathbf{w}_1 \end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix} 1 \\ \mathbf{x}_1 \end{bmatrix}$ (both $(D+1) \times 1$).

4 Logistic Regression

Linear model is:

$$z = \mathbf{w}^T \mathbf{x}$$

Prediction is:

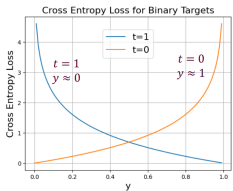
$$y = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

For which the loss functions are:

- Zero-One Loss: $L_{0-1}(y, t) = \mathbb{I}[y \neq t]$
– Non-differentiable / zero gradients (cannot update weights).
- Squared Error Loss: $L_{SE}(z, t) = \frac{1}{2}(z - t)^2$
– Penalizes confident correct predictions.
- Sigmoid + Squared Error:
 $y = \sigma(z) = \frac{1}{1+e^{-z}}$, $L_{SE}(y, t) = \frac{1}{2}(y - t)^2$
– Suffers from vanishing gradients (when predictions are extremely wrong).
- Cross entropy: $\begin{cases} -\log(y) & t = 1 \\ -\log(1 - y) & t = 0 \end{cases}$

Where cross entropy can be:

$$-t \log(y) - (1 - t) \log(1 - y)$$



4.1 Cross-Entropy Loss Equations

Linear model

$$z = \mathbf{w}^T \mathbf{x}$$

Classification

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\frac{\partial y}{\partial z} = y(1 - y)$$

Loss

$$\mathcal{L}_{CE}(y, t) = -t \log(y) - (1 - t) \log(1 - y)$$

Gradient Descent Update

Where $\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{CE}(y^{(i)}, t^{(i)})$

$$\frac{\partial \mathcal{L}_{CE}}{\partial w_j} = \frac{\partial \mathcal{L}_{CE}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

$$= \left(-\frac{t}{y} + \frac{1-t}{1-y} \right) \cdot y(1-y) \cdot x_j$$

$$= (y - t)x_j$$

Tip: re-use variables, don't sub all!

The gradient descent update for logistic regression is:

$$w_j \leftarrow w_j - \alpha \frac{\partial \mathcal{E}}{\partial w_j}$$

$$= w_j - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{t})$$